SSH-C2-Broker

Centralized Access for Remote Appliances

Jesse Rodriguez
Principal Security Consultant
DirectDefense Inc.

github.com/DirectDefense/ssh-c2-broker

Why We Built It

- **COVID changed everything**
  - Remote assessments became the norm overnight
- **Commercial tools are expensive**
  - Require infrastructure you don’t own or control
- **Segmentation matters**
  - Compromise of one appliance must not pivot to other client networks

The Solution

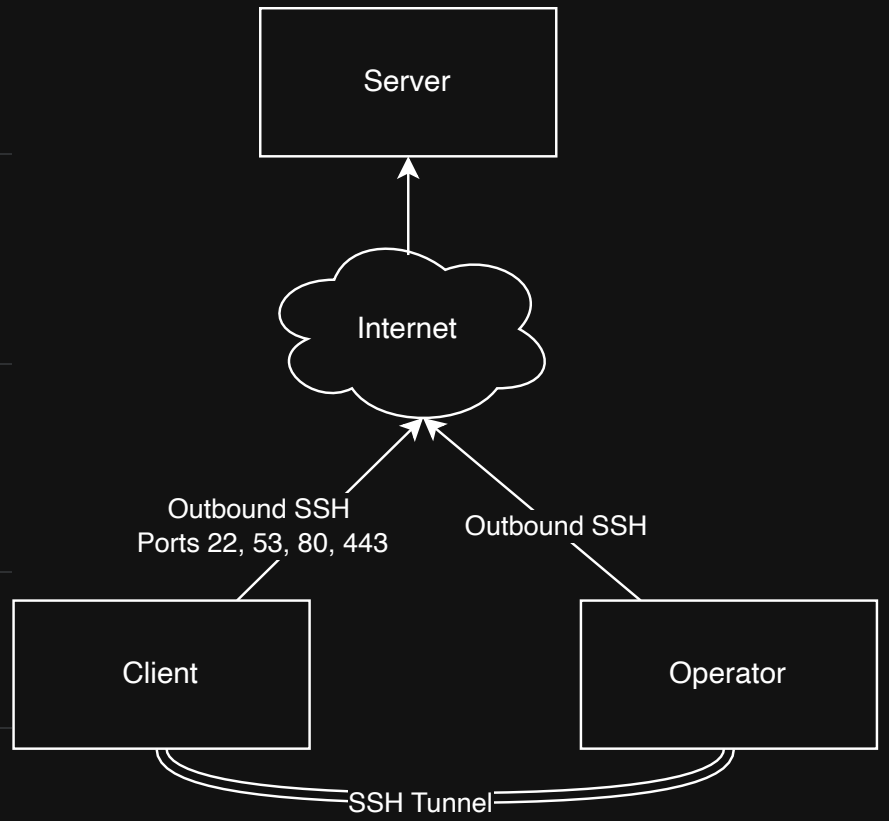


AKA

SSH-C2-Broker

Architecture

Component	Role	Function
Client	Target machine	Maintains reverse tunnel + SOCKS5 proxy back to broker
Server	Relay node	Assigns ports, tracks registered devices, mediates sessions
Operator	Red teamer	Connects through broker to any registered device



The Client

- Registers itself to the server on install & receives a persistent port number (e.g. `30001`)
- Maintains a reverse tunnel via `autossh` on that persistent port
- Multi-port fallback: tries `22`, `53`, `80`, `443` to reach the server
- SOCKS5 proxy on `127.0.0.1:9050` for proxied traffic to the Internet (proxychains)
- Cron heartbeat every 2 minutes via `callback.sh`

```
# Client/callback.sh
autossh \
  -i $key -p $port $username@$ip \
  -M 0 \
  -R $port_num:localhost:22 \
  -D 127.0.0.1:9050 \
  -f -T -N \
  -o "ServerAliveInterval=30" \
  -o "ServerAliveCountMax=3" \
  -o "StrictHostKeyChecking=no"
```

The Server (Broker)

- Listens on ports `22`, `53`, `80`, `443`
- Uses restricted `c2` user with `rbash`
- Device database: `port,hostname,tag` in `/home/c2/c2database.txt`
- Scripts in `/usr/bin/` — `registerc2.sh`, `active_c2.sh`, `active_c2_tagged.sh`
- **Authentication not handled via installer**

Device Registration

- On first connect, the client SSH's to the server and calls `registerc2.sh <hostname> <tag>`
- If the hostname already exists in the database, the existing persistent port is returned
- If the hostname does not exist, the next persistent port is assigned
- The device can be registered to a new C2 server by changing `c2_ip.txt` and reinstalling

The Operator Interface

1. SSH to broker, call `active_c2_tagged.sh`, receive live device list
2. Select a device via the interactive menu, or pass port number as `$1` to skip the prompt
3. Maps the port with `ssh -L <port>:localhost:<port>`
4. Start SOCKS5 proxy with `ssh -D <port+10000>`
5. Drop into shell: `ssh -p <port> localhost`
6. On exit, the map and proxy background PIDs are killed automatically

```
>./c2connect.sh
Available Devices:
30001 - web-host   - dc-dmz
30002 - jump-host  - internal
30003 - db-server  - internal

>Device: 30001

Initializing Map... Map Complete
Proxy Successful  (SOCKS5 on localhost:40001)
Dropping into shell...
admin@web-host:~$
```


LIVE DEMO TIME

Key Takeaways

Security

- Authentication via OpenSSH (use Signed Certs)
- Compromise of one client or the server broker should not compromise others
- Clients cannot route to other clients
- You own all the infrastructure

Capabilities

- Multi-port fallback (22/53/80/443)
- Per-target SOCKS5 proxy on `port+10000` for proxied web/tool traffic
- SOCKS5 proxy via server for proxychains
- Built on OpenSSH

🔗 `github.com/DirectDefense/ssh-c2-broker`

Questions?

Jesse Rodriguez
Principal Security Consultant
DirectDefense Inc.
jesse@directdefense.com
Linkedin.com/in/jrodriguez127
Discord/X: @ShadyBrownGuy
GitHub: @JRodriguez556

github.com/DirectDefense/ssh-c2-broker

with thanks to:
Alton Johnson
Mark Beard
Nick Shuman
Dean Meyer

⚠ Authorized use only — deploy only against systems you own or have explicit written permission to test.